Purpose

Overview of the customer facing JSON API for the Hobart Online Ordering system. Exact and complete specification is provided in additional documentation.

Summary

A REST-based API communicating via JSON objects is the primary interface to the Hobart Online Ordering system. The primary data structures used in this system describe orders, items within orders, and customary Hobart products. Endpoints are provided to add, query, modify, and delete each of these. Additional endpoints are provided to support this purpose, including categories, departments, operators, and images. An OpenAPI specification is provided to describe the exact details of the API.

API

The Online Ordering system provides a REST-based API, which sends and receives data as JSON. Endpoints are located under `/api/v1/`. An OpenAPI document provides a specification of all public endpoints, along with schemas for all common objects required for input and output. Generally, new items of any type can be added with a POST, modified with a PUT, fetched with a GET, and removed with a DELETE. A 200 response indicates success, while a 400 response indicates an invalid input. In most cases, unused fields may be left null.

The Online Ordering OpenAPI specification is provided as a YAML file, which may be examined either through Visual Studio Code with the “OpenAPI extension for Visual Studio Code” (preferred), directly with any text editor, with the Swagger editor, or by viewing the provided HTML document generated from the specification file.

Orders and Order Details

Orders are represented as one `order` with zero or more `orderdetails`. Orders are associated with a department; each `order` is uniquely identified by the pair (department number, order number). Each `order` contains data such as the requested pickup time, customer personal information such as name and phone number, and order status. Orders returned by the API may be constrained with a time range, limited to a certain order status, sorted by any field, or limited to a certain number of results. For example, the following request would return all orders that have not been started, due on November 10th 2021, ordered by pickup time ascending:

```
/api/v1/orders?status=1&sort=OrderPickup&dir=asc&after=2021-11-
10%2000:00:00&before=2021-11-11%2023:59:59
```

Each `orderdetails` represents a single product in the order and is uniquely represented by the triplet (department number, order number, orderdetails number). An `orderdetails` contains information such as PLU number, requested quantity or weight, preparation instructions, and item status. Each `orderdetails` item may be filled, substituted, or cancelled (depending on system configuration and customer instructions, substitutions may or may not be allowed). The endpoints for

`orderdetails` can be queried to return a specific `orderdetails` or all `orderdetails` within a specified order. For example, all `orderdetails` for order 1 of department 0 can be queried at:

```
/api/v1/orderdetails/0/1
```

Products, Operators, Departments, and Categories

Configuration of products, operators, departments, and categories is required for the Online Ordering system to function. It is required that all products, departments, and categories must be kept in sync between the Online Ordering system and all scales accessing it. If this data does not match exactly, orders may not be able to be filled correctly. For ease of interoperation, the Online Ordering system can receive and report this information via either the legacy Hobart Tag interface or via this JSON API. The Online Ordering system represents these in a consistent manner with existing Hobart equipment. The API allows these to be managed individually and in bulk. A brief overview of the fields most relevant for Online Ordering follows.

Products
`PrNum`: product number.
`DeptNum`: department number.
`CaNum`: category number.
`PrPrice`: product price.
`PrType`: product type.
`PrDesc`: product description.

Operators
`OpNum`: operator number.
`OpAccLev`: operator access level.

Departments
`DeptNum`: department number.
`DeptName1`: department name.

Categories
`DeptNum`: department number
`CaNum`: category number.
`CaName1`: category name.

Time

All times required and reported by the API must be in the local time that the Online Ordering system is configured for. Times must be in the format YYYY-MM-DD HH:MM:SS. Hours are based on the 24-hour system, i.e., the first hour of the day is 0, and the last hour of the day is 23.

Images

Endpoints to manage images for products and categories are provided. These images will be displayed on the upcoming Hobart kiosk solution, which will interoperate with this Online Ordering system, but are not required for the core functionality of the Online Ordering system. Images are sent as base64 encoded files, with a small amount of metadata. Acceptable image formats are JPEG, GIF, and PNG. Files will be used as provided; for responsive user functionality with minimal latency, use compressed images with the smallest acceptable size.

Usage Examples

The API may be readily used with the `curl` utility. Web browsers also provide a convenient interface to read from the API and pretty-print JSON. Some example commands are provided below; modify the address as needed.

Fetch all orders which have not been started:

```
curl 10.3.128.104/api/v1/orders/?status=1
```

Add a new order to department 1 (minimum fields included):

```
curl -X POST 10.3.128.104/api/v1/orders -H ‘Content-Type:
application/json’ -d ‘[{“DeptNum”: 1, “OrderNumber”: 1, “OrderStatus”:
1, “CustomerName”: “Example Customer”, “OrderOrigin”: 1,
“OrderPickup”: “2021-11-10 14:30:00”}]’
```

Delete all products from the system:

```
curl -X DELETE 10.3.128.104/api/v1/products/all
```

Upload a new product image (requires the base64 program):

```
(echo -n '{"filename":"newProd.png", "image":"'; base64
./newProd.png ; echo '"}') | curl -H "Content-Type: application/json"
-d @- 10.3.128.81/api/v1/images/products/
```

Example Order Workflow

A customer wishes to purchase a loaf of rye bread and 1¾ pounds of ham. Because these items are in different departments, two orders are created, one for the meat department, and the other for the bakery.

First, JSON similar to the following is POSTed to /api/v1/orders to create the orders:

```
{"DeptNum": 0, "OrderNumber": 456, "OrderStatus": 1, "CustomerName":
"Imaginary Customer", "OrderOrigin": 1, "OrderCreation": "2022-01-20
07:20:00", "OrderPickup": "2022-01-20 10:30:00", "AllowSubstitutions":
1}

{"DeptNum": 1, "OrderNumber": 456, "OrderStatus": 1, "CustomerName":
"Imaginary Customer", "OrderOrigin": 1, "OrderCreation": "2022-01-20
07:20:00", "OrderPickup": "2022-01-20 10:30:00", "AllowSubstitutions":
1}
```

Then JSON similar to the following is POSTed to /api/v1/orderdetails/ to add the items:

```
{"DeptNum": 0, "OrderNumId": 0, "OrderNumber": 456, "PluNumber": 123,
"PluByCount": 1}

{"DeptNum": 1, "OrderNumId": 0, "OrderNumber": 456, "PluNumber": 789,
"PluWeight": 1750, "PluInstructions": "sliced as thin as possible"}
```

The rye loaf order is then shown on any scales configured in the bakery, and the ham in the meat department. The rye loaf is available, so the bakery operator fills it (`Filled` is set to 1 in the `orderdetails`), and the order is marked complete (`OrderStatus` is set to 3). The ham is unavailable, so the meat department operator marks it as cancelled, giving the reason “out of ham” (`Cancelled` set to 1, `ReasonCancelled` set to “out of ham”); then the order is marked as complete (`OrderStatus` is set to 3).

If needed, final order status and actual amounts filled can then be polled via the API. Customized integration with external systems may be possible; contact Hobart for details.

Revision History

11/10/2021 Initial draft (EMC)
02/09/2022 Add example order flow and reword image API description (EMC)